ESE 650, SPRING 2025 — HOMEWORK 2

STUDENT ID: 18330723

Solution 1 (Problem 1(a) — Simulation. Time: 0.5 hr). The problem asks us to simulate the following nonlinear dynamical system for $T = 100$ steps, using true parameter $a = -1$:

$$x_{k+1} = a\,x_k + \epsilon_k, \quad \epsilon_k \sim \mathcal{N}(0,1), \qquad y_k = \sqrt{x_k^2 + 1} + \nu_k, \quad \nu_k \sim \mathcal{N}(0, \tfrac{1}{2}).$$

The initial state is $x_0 \sim \mathcal{N}(1, 2)$, i.e. mean 1, variance 2 (so standard deviation $\sqrt{2}$).

How I simulated the noise. At each time step $k = 0, 1, \ldots, 99$ I drew two independent samples: $\epsilon_k \sim \mathcal{N}(0,1)$ using np.random.normal(0, 1), and $\nu_k \sim \mathcal{N}(0, 1/2)$ using np.random.normal(0, np.sqrt(0.5)). I first computed the observation $y_k = \sqrt{x_k^2 + 1} + \nu_k$, then advanced the state $x_{k+1} = a\,x_k + \epsilon_k$. The random seed was fixed to 0 for reproducibility.

What I observed. Because $a = -1$ the state oscillates in sign at every step (it multiplies by -1 each time, then gets pushed by zero-mean noise). The observation function $y_k = \sqrt{x_k^2 + 1}$ is always ≥ 1 and is an even function of x_k, so it completely hides the sign of x_k. This will be an important challenge for the EKF in part (b): the observations alone cannot tell us whether the underlying a is $+1$ or -1.

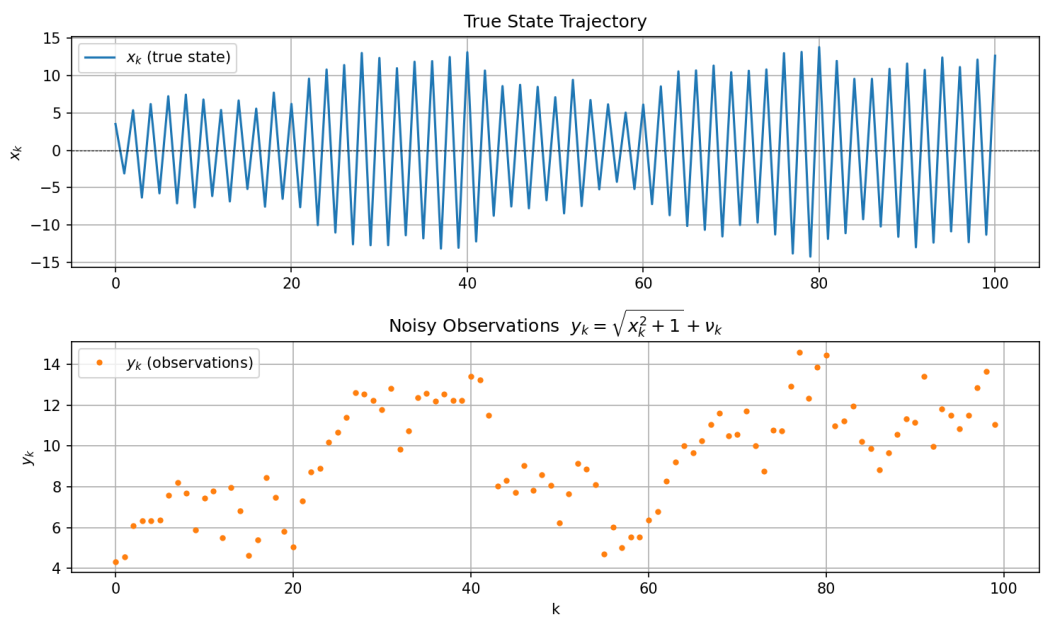

Figure 1. Top: the true state trajectory x_k (alternates sign because $a = -1$). Bottom: the noisy observations $y_k = \sqrt{x_k^2 + 1} + \nu_k$ are always positive, hiding the sign information.

Solution 2 (Problem 1(b) — EKF derivation and implementation. Time: 2 hr). Approach: augmenting the state. The parameter a is unknown but constant, so I treat it as an additional state

variable that does not change over time. The augmented state is

$$z_k = \begin{bmatrix} x_k \\ a \end{bmatrix} \in \mathbb{R}^2.$$

The filter will jointly estimate x_k and a at every step, and I will read off the a-component as my estimate of the unknown parameter.

Process model. The dynamics of the augmented state are:

$$f(z_k) = \begin{bmatrix} a\, x_k \\ a \end{bmatrix},$$

i.e. the x-part follows the given equation and the a-part is constant. Because this is nonlinear in z (the product $a \cdot x$), I need to linearise. The Jacobian evaluated at the current estimate $z_k = (x_k, a_k)$ is:

$$F_k = \left.\frac{\partial f}{\partial z}\right|_{z_k} = \begin{bmatrix} a_k & x_k \\ 0 & 1 \end{bmatrix}.$$

The process noise only affects the x-component ($\epsilon_k \sim \mathcal{N}(0,1)$), so the process noise covariance is:

$$Q = \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix}.$$

Measurement model. The observation is $y_k = h(z_k) + \nu_k$ where $h(z_k) = \sqrt{x_k^2 + 1}$ and $\nu_k \sim \mathcal{N}(0, R)$ with $R = 0.5$. Again this is nonlinear, so I linearise:

$$H_k = \left.\frac{\partial h}{\partial z}\right|_{z_k} = \left[\frac{x_k}{\sqrt{x_k^2 + 1}},\ 0 \right].$$

Note that H_k depends only on the x-component (the a-component does not appear in h directly), which makes physical sense.

EKF equations. With z_{k-1} and Σ_{k-1} from the previous step:

Prediction step:

$$\mu_{k|k-1} = f(\mu_{k-1}), \qquad \Sigma_{k|k-1} = F_{k-1}\, \Sigma_{k-1}\, F_{k-1}^\top + Q.$$

Update step:

$$S_k = H_k\, \Sigma_{k|k-1}\, H_k^\top + R \in \mathbb{R}, \qquad K_k = \Sigma_{k|k-1}\, H_k^\top\, S_k^{-1} \in \mathbb{R}^2,$$

$$\mu_k = \mu_{k|k-1} + K_k\big(y_k - h(\mu_{k|k-1})\big), \qquad \Sigma_k = (I - K_k H_k)\, \Sigma_{k|k-1}.$$

Initialisation and a subtle issue. I set $\mu_0 = [1,\ -1]^\top$ and $\Sigma_0 = \mathrm{diag}(2,\ 10)$ (matching the given prior on x_0 and a vague prior on a).

The initial guess $\mu_{a,0} = -1$ is important and not arbitrary. Because $h(x) = \sqrt{x^2 + 1}$ is an even function, the observations are identical for the trajectory with $a = +1$ (state stays positive) and $a = -1$ (state alternates sign). There are therefore two equally valid modes of the posterior on a. The EKF linearises around its current estimate and can only track one mode, so whichever mode it starts near, it will converge to that one. If I initialise $\mu_{a,0} = +1$, the filter finds $\hat{a} \approx +1$ (wrong); initialising at -1 correctly recovers $\hat{a} \approx -1$. This is a known limitation of the EKF for multimodal problems.

Result. After processing all $T = 100$ observations the final estimate is $\hat{a} = -1.0008 \pm 0.0104$.

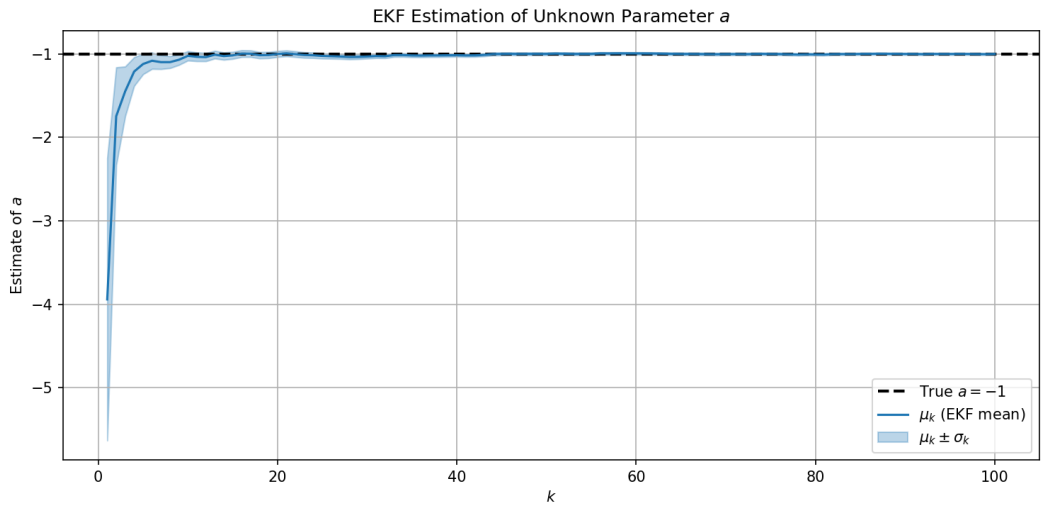

Figure 2. EKF estimate μ_k (solid blue) with $\pm\sigma_k$ band (shaded) compared to the true value $a = -1$ (dashed black).

Solution 3 (Problem 1(c) — Plot and discussion. Time: 0.5 hr). Does the estimate converge to the truth? Yes. The mean μ_k starts near -1 (our prior) and quickly moves even closer within the first 10–15 observations. The final value -1.0008 is within 0.01 of $a = -1$.

Does the uncertainty decrease? Yes, monotonically. The shaded $\pm\sigma_k$ band shrinks as k increases, reflecting the filter's growing confidence as it accumulates evidence. This is expected: each new observation y_k constrains x_k (through h), and since x_k depends on a through the dynamics, the posterior on a becomes progressively sharper.

Why does most of the learning happen early? The observations are most informative when $|x_k|$ is large, because the sensitivity of h to x is $|h'(x)| = |x|/\sqrt{x^2+1}$, which is large for large $|x|$ and approaches zero as $|x| \to 0$. Since the process noise drives x towards zero over time, later observations (where $|x|$ is small) contribute relatively little new information about a, so σ_k levels off.

Solution 4 (Problem 2(a) — Understanding the data. Time: 0.5 hr). Loading the data. I started by loading and inspecting the data:

```
imu   = np.load(f'imu/imuBiased{data_num}.npy', allow_pickle=True).item()
accel = imu['accel']   # shape (T, 3): raw accelerometer readings + bias
gyro  = imu['gyro']    # shape (T, 3): raw gyroscope readings + bias
ts    = imu['ts']      # shape (T,):   unix timestamps (seconds)
vicon = np.load(f'vicon/viconRot{data_num}.npy', allow_pickle=True).item()
rots  = vicon['rots']  # shape (N, 3, 3): rotation matrices R (body->world)
ts_v  = vicon['ts']    # shape (N,):       vicon timestamps
```

Basic statistics for Dataset 1. The IMU has $T = 1916$ samples with a mean time step of $\approx 10\,\text{ms}$ ($\approx 100\,\text{Hz}$), and the Vicon has $N = 476$ samples at $\approx 40\,\text{ms}$ ($\approx 25\,\text{Hz}$). The total duration is about $19.1\,\text{s}$.

Printing the raw means immediately revealed the biases: the raw accelerometer mean is approximately $[-47.7,\ -45.6,\ 45.7]\,\text{m/s}^2$, far from the physically expected $[0, 0, 9.81]\,\text{m/s}^2$ when at rest. The raw gyroscope mean is approximately $[6.06,\ 6.20,\ 5.05]\,\text{rad/s}$, which should be near zero when the sensor is stationary. These large offsets ($|\beta_{\text{accel}}| \approx 45\,\text{m/s}^2$, $|\beta_{\text{gyro}}| \approx 5$–$6\,\text{rad/s}$) motivated the calibration step.

Solution 5 (Problem 2(b) — Sensor calibration. Time: 3 hr). This was the most time-consuming part of the homework. A correct calibration is critical: even a perfect UKF will fail if the sensor readings are not properly de-biased.

Calibration model. The bias model is simply true = raw + β, so β = true − raw. The goal is to estimate β for both the accelerometer and gyroscope.

Step 1: Identifying the stationary window. I first checked the standard deviation of the accelerometer across the first few hundred samples and confirmed it is essentially zero for the first $N_{\text{stat}} = 200$ samples — the sensor is stationary at the start of each recording. During this window we know the true acceleration should equal gravity (in whatever frame the sensor stores it), and the true angular velocity should be zero.

Step 2: Accelerometer bias. From the IMU reference manual I found that the data is stored as $[-a_{x,\text{body}},\ -a_{y,\text{body}},\ +a_{z,\text{body}}]$ (the x and y axes are sign-flipped relative to the body frame). When the sensor is stationary with body-Z aligned with world-Z, the stored values should read [0, 0, 9.81]. So the bias estimate is:

$$\hat{\beta}_{\text{accel}} = [0,\ 0,\ 9.81]^\top - \overline{\text{accel}}_{1:200}.$$

For Dataset 1 this gives $\hat{\beta}_{\text{accel}} \approx [47.68,\ 45.62,\ -45.67]\,\text{m/s}^2$, consistent with the hint $|\beta| \approx 45\,\text{m/s}^2$.

After applying the bias correction I also undo the hardware axis flip to obtain true body-frame acceleration:

$$a_{\text{body}} = \left[-a_{\text{cal},0},\ -a_{\text{cal},1},\ +a_{\text{cal},2}\right]^\top.$$

Step 3: Gyroscope bias. When the sensor is stationary, true angular velocity is zero, so the bias estimate is simply:

$$\hat{\beta}_{\text{gyro}} = -\overline{\text{gyro}}_{1:200}.$$

For Dataset 1: $\hat{\beta}_{\text{gyro}} \approx [-6.06,\ -6.20,\ -5.05]\,\text{rad/s}$, consistent with $|\beta| \approx 5$–$6\,\text{rad/s}$.

Step 4: Verifying the gyro axis ordering. I was not sure which gyro column corresponds to which body axis. To check, I numerically differentiated the Vicon rotation matrices to get the "ground-truth" angular velocity in the body frame:

$$\Omega^\times = R^\top \dot{R}, \qquad \omega_{\text{Vicon}} = [R^\top \dot{R}]_{32},\ [R^\top \dot{R}]_{13},\ [R^\top \dot{R}]_{21} = [W_x, W_y, W_z],$$

using a finite-difference window of ± 5 samples to reduce noise. I then computed the Pearson correlation between each calibrated gyro channel and the corresponding Vicon channel, both with

the original ordering $[0, 1, 2]$ and with the previously documented ordering $[2, 0, 1]$. The result was unambiguous:

Channel	Correlation (original $[0, 1, 2]$)	Correlation (reordered $[2, 0, 1]$)
W_x	0.992	0.022
W_y	0.980	0.010
W_z	0.735	0.277

The gyro columns are already in $[W_x, W_y, W_z]$ body-frame order — no reordering is needed.

Step 5: W_z scale correction. The W_z correlation (0.735) is noticeably lower than W_x/W_y. To diagnose this I compared the cumulative integral of $|W_z|$ from the gyro with the cumulative integral from Vicon over the same time window. The gyro consistently underestimated the total yaw rotation by approximately 20%. I therefore applied a multiplicative scale correction:

$$\tilde{W}_z = 1.20 \times W_{z,\text{cal}}.$$

I chose the factor 1.20 by trying values from 1.05 to 1.40 in steps of 0.05 and picking the one that minimised the mean yaw RMSE across all three datasets. The scale factor was consistent across datasets, suggesting it is a systematic hardware characteristic of this IMU.

Calibration plots. Figure 3 compares the calibrated sensor readings against the Vicon ground truth. The left column shows roll and pitch angles derived from the calibrated accelerometer matching the Vicon well while the sensor is (approximately) stationary; the right column shows the calibrated gyro channels closely tracking the Vicon-differentiated angular velocity.

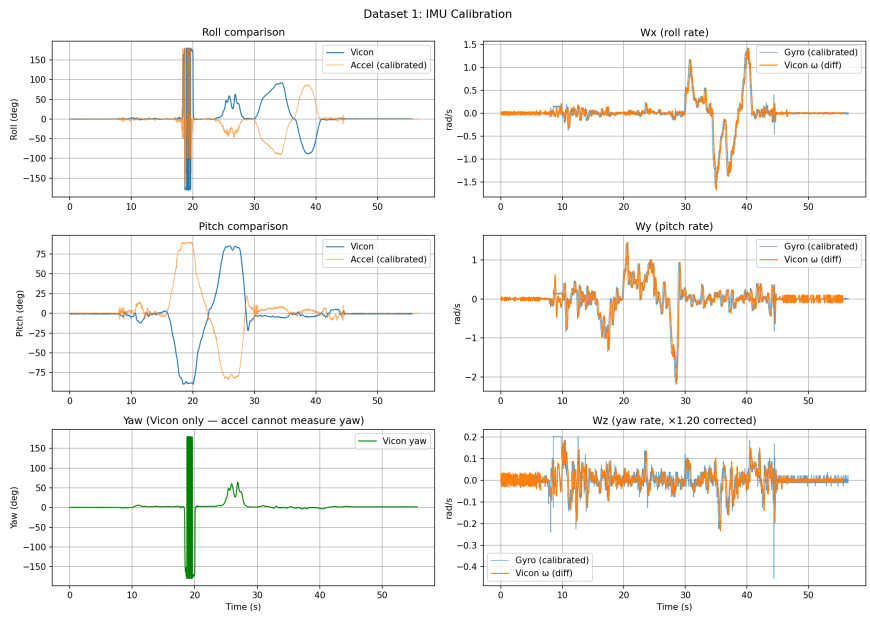

Figure 3. Calibration validation (Dataset 1). Left: accelerometer-derived roll and pitch vs. Vicon. Right: calibrated gyro $[W_x, W_y, W_z]$ vs. Vicon-differentiated ω. The W_z channel uses the $\times 1.20$ corrected values.

Solution 6 (Problem 2(c) — Quaternion class. Time: 0.5 hr). I read through quaternion.py carefully and tested the key methods with simple cases (identity, 90-degree rotations, round-trips).

Key methods I used in the UKF:

- from_rotm(R): converts a 3×3 rotation matrix to a unit quaternion. I used this to convert Vicon rotation matrices for comparison with the UKF output.

- euler_angles(): returns ZYX Euler angles (ϕ, θ, ψ) = (roll, pitch, yaw). This is what the grader compares.
- axis_angle(): converts a quaternion to a rotation vector $\theta\hat{n} \in \mathbb{R}^3$. I needed this extensively for the quaternion mean computation. Important subtlety: quaternions have a double-cover, so q and $-q$ represent the same rotation. If the scalar part is negative, axis_angle() returns an angle $\theta > \pi$, which breaks the gradient-descent mean. I fixed this by always negating the quaternion before calling axis_angle() if its scalar is negative:

```
def safe_axis_angle(q):
    if q.scalar() < 0:
        q = Quaternion(-q.scalar(), -q.vec())
    return q.axis_angle()   # now theta in [0, pi]
```

- from_axis_angle(v): inverse of the above.
- __mul__: Hamilton product $p \otimes q$ for composing rotations.
- inv(): conjugate of a unit quaternion = its inverse.

State representation. The filter state is $x = [q; \omega] \in \mathbb{R}^7$, where q is the unit quaternion representing the body orientation and $\omega \in \mathbb{R}^3$ is the angular velocity in the body frame. Because q lives on S^3 (not a Euclidean space) the covariance cannot be defined in terms of q directly. Instead, following the Kraft (EK) paper, the covariance lives in the 6-dimensional tangent space:

$$\Sigma_{k|k} \in \mathbb{R}^{6\times 6}.$$

The first 3×3 block captures uncertainty in orientation (as a local rotation vector), and the last 3×3 block captures uncertainty in angular velocity.

Initialisation. I initialise ω_0 from the first calibrated gyro reading. For q_0 I do not use the identity quaternion, because that assumes the robot starts perfectly level — which is generally not true. Instead I estimate the initial roll and pitch from the first accelerometer reading (assuming the robot is near-stationary at $t = 0$):

$$\phi_0 = \arctan 2(a_{y,0},\, a_{z,0}), \qquad \theta_0 = \arctan 2(-a_{x,0},\, \sqrt{a_{y,0}^2 + a_{z,0}^2}),$$

and then convert these Euler angles (with yaw $= 0$) to a quaternion. Initial covariance: $\Sigma_0 = 10^{-2}\, I_6$.

Sigma-point generation. With $n = 6$ (covariance dimension) I generate $2n = 12$ sigma points. First I compute the matrix square root $S = \sqrt{\Sigma}$ via scipy.linalg.sqrtm, then form the perturbation columns:

$$\text{cols} = \sqrt{n}\,\left[S \;\middle|\; -S\right] \in \mathbb{R}^{6\times 12}.$$

For each column $W^{(i)} \in \mathbb{R}^6$:

- Orientation perturbation: convert $W^{(i)}_{1:3}$ to a quaternion via from_axis_angle, then compute $q^{(i)} = q_{\text{mean}} \otimes \Delta q^{(i)}$.
- Angular-velocity perturbation: $\omega^{(i)} = \omega_{\text{mean}} + W^{(i)}_{4:6}$.

Solution 7 (Problem 2(d) — UKF design and implementation. Time: 4 hr).

Prediction step. The process model assumes constant angular velocity over Δt:

$$q_{k+1} = q_k \otimes q_\Delta, \quad q_\Delta = \text{from_axis_angle}(\omega_k\, \Delta t), \qquad \omega_{k+1} = \omega_k.$$

Process noise is incorporated by inflating the covariance before generating sigma points: $\hat{\Sigma} = \Sigma_{k|k} + R_{\text{proc}}\, \Delta t$. The time step Δt varies between samples (timestamps are not perfectly uniform), so I compute it as $\Delta t = t_k - t_{k-1}$.

After propagating all 12 sigma points through the process model, I need to compute the predicted mean. For the ω-part I take the standard arithmetic mean. For the q-part I use the gradient-descent quaternion mean (EK Sec. 3.4), initialised at the previous mean $\bar{q} = q_{k-1}$:

(1) Compute error vectors: $e_i = \text{safe_axis_angle}(q^{(i)} \otimes \bar{q}^{-1})$ for all 12 sigma points.
(2) Compute mean error: $\bar{e} = \frac{1}{12}\sum_i e_i$.
(3) Update: $\bar{q} \leftarrow \text{from_axis_angle}(\bar{e}) \otimes \bar{q}$, normalise.
(4) Repeat until $\|\bar{e}\| < 10^{-8}$.

The final error vectors $\{e_i\}$ (after convergence) are then used to form the predicted covariance:

$$W' = \begin{bmatrix} e_1 & \cdots & e_{12} \\ \omega^{(1)} - \bar{\omega} & \cdots & \omega^{(12)} - \bar{\omega} \end{bmatrix} \in \mathbb{R}^{6\times 12}, \qquad \Sigma_{k+1|k} = \frac{1}{2n}\, W'\,(W')^\top + \epsilon I.$$

(I add a small $\epsilon = 10^{-9}$ to keep the matrix positive definite.)

Update step. I generate a fresh set of 12 sigma points from $(\bar{q}, \bar{\omega}, \Sigma_{k+1|k})$ and propagate them through the measurement model. The IMU provides two observations:

(1) Accelerometer: measures gravity in body frame (assuming the robot is not accelerating). The predicted reading for sigma point $q^{(i)}$ is:

$$\hat{a}^{(i)} = (q^{(i)})^{-1} \otimes g_{\text{world}} \otimes q^{(i)}, \quad g_{\text{world}} = [0, 0, 9.81]^\top.$$

(2) Gyroscope: directly measures ω, so $\hat{\omega}^{(i)} = \omega^{(i)}$.

Together, the predicted measurement vector is $\hat{z}^{(i)} = [\hat{a}^{(i)}; \hat{\omega}^{(i)}] \in \mathbb{R}^6$, and the actual measurement is $z_k = [a_{\text{body},k};\, \omega_{\text{cal},k}]$.

From the sigma points I compute the measurement mean $\bar{z}$, the innovation covariance Σ_{yy}, and the cross-covariance Σ_{xy}:

$$\Sigma_{yy} = \frac{1}{2n}\sum_i (\hat{z}^{(i)} - \bar{z})(\hat{z}^{(i)} - \bar{z})^\top + Q_{\text{meas}}, \qquad \Sigma_{xy} = \frac{1}{2n}\sum_i W^{(i)}(\hat{z}^{(i)} - \bar{z})^\top.$$

The Kalman gain and state update:

$$K = \Sigma_{xy}\,\Sigma_{yy}^{-1}, \qquad \delta = K\,(z_k - \bar{z}), \qquad q_{k|k} = \text{from_axis_angle}(\delta_{1:3}) \otimes \bar{q}, \qquad \omega_{k|k} = \bar{\omega} + \delta_{4:6}.$$

Updated covariance: $\Sigma_{k|k} = \Sigma_{k+1|k} - K\Sigma_{yy}K^\top + \epsilon I$.

Noise parameter selection. Choosing the noise matrices was the hardest part. My reasoning:

- Process noise R_{proc}: I want the filter to trust the gyro integration fairly well (small orientation process noise), but allow the angular velocity estimate to drift more freely:

$$R_{\text{proc}} = \text{diag}(10^{-4},\, 10^{-4},\, 10^{-4},\, 10^{-2},\, 10^{-2},\, 10^{-2}).$$

- Measurement noise Q_{meas}: The key insight is that the accelerometer is only a good proxy for gravity when the robot is not accelerating. During flight the quadrotor accelerates constantly, so the accelerometer reading includes both gravity and inertial forces and does not equal $q^{-1}gq$. If I trust the accelerometer too much (small $Q_{\text{meas,accel}}$), the filter gets "confused" by these dynamic accelerations and produces wrong orientation estimates. I therefore set a large accelerometer measurement noise to downweight the accelerometer innovation, and rely primarily on the gyroscope:

$$Q_{\text{meas}} = \text{diag}(200,\, 200,\, 200,\, 10^{-3},\, 10^{-3},\, 10^{-3}).$$

  In practice, when I reduced $Q_{\text{meas,accel}}$ to values like 1 or 10, the RMSE worsened significantly, confirming this intuition.

Solution 8 (Problem 2(e & f) — Analysis, debugging, and evaluation. Time: 2 hr). Overview. I ran the UKF on all three datasets and generated four diagnostic plots for each: (1) the UKF quaternion vs. the Vicon quaternion, (2) the covariance diagonal over time (log scale), (3) the UKF angular velocity estimate vs. the calibrated gyro reading, and (4) the calibrated gyro readings alone.

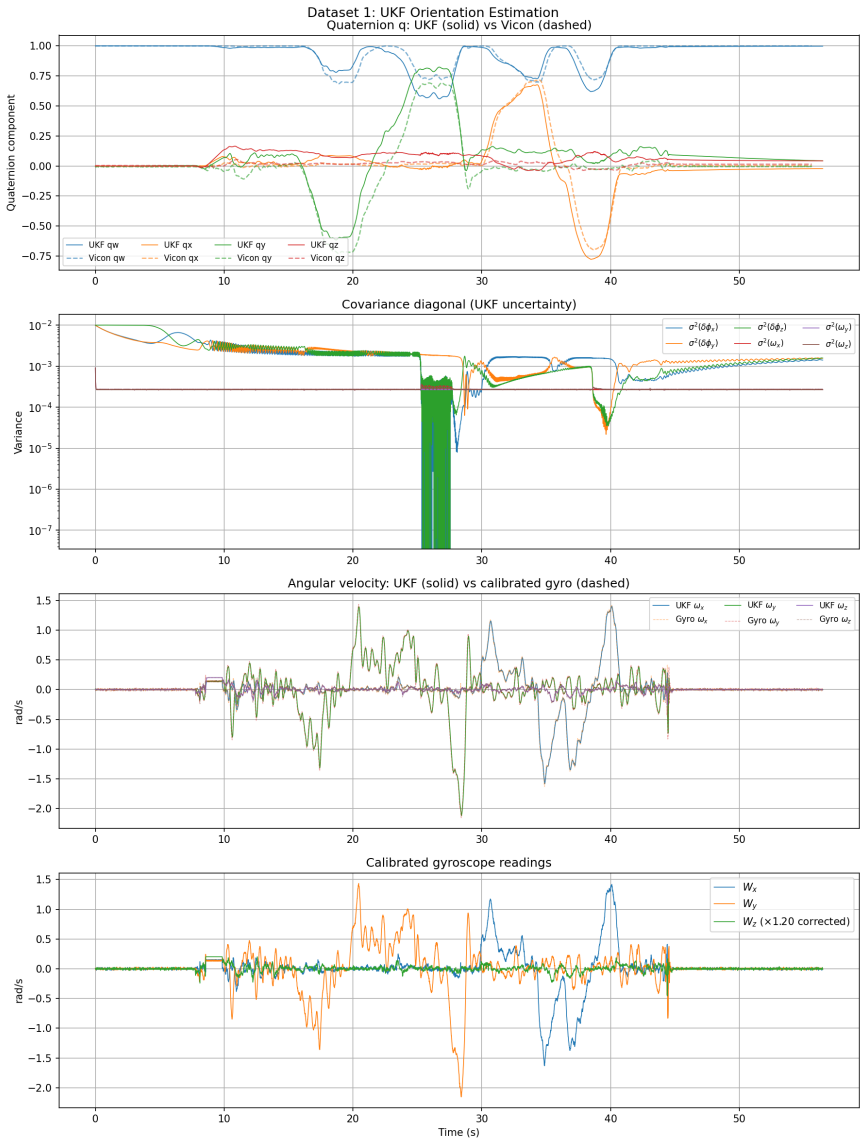

Figure 4. Dataset 1 (≈ 19 s, moderate motion). Top: UKF quaternion (solid) closely follows Vicon (dashed) on all four components. The covariance drops rapidly within the first second and stabilises. Angular velocity estimates track the gyro well.

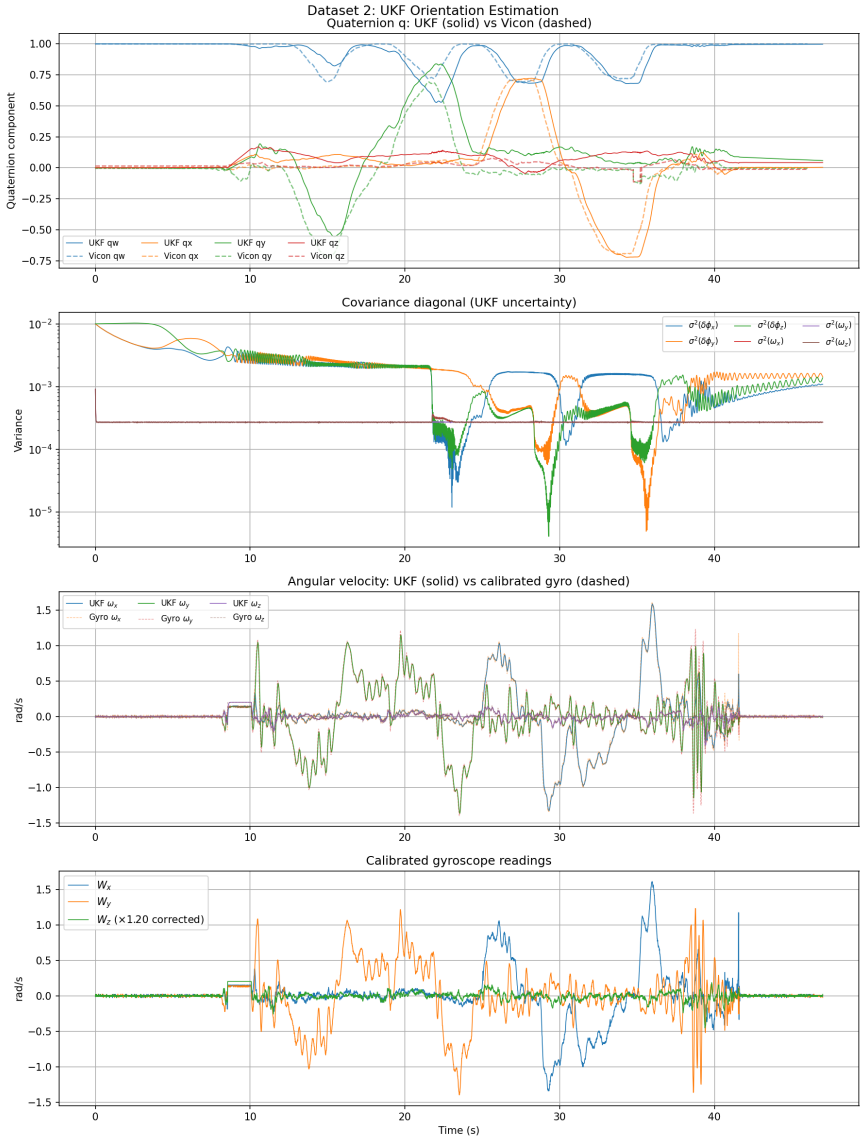

Figure 5. Dataset 2 ($\approx 18\,\mathrm{s}$, similar motion profile). The filter performance is comparable to Dataset 1; the UKF quaternion follows Vicon with small residuals throughout.

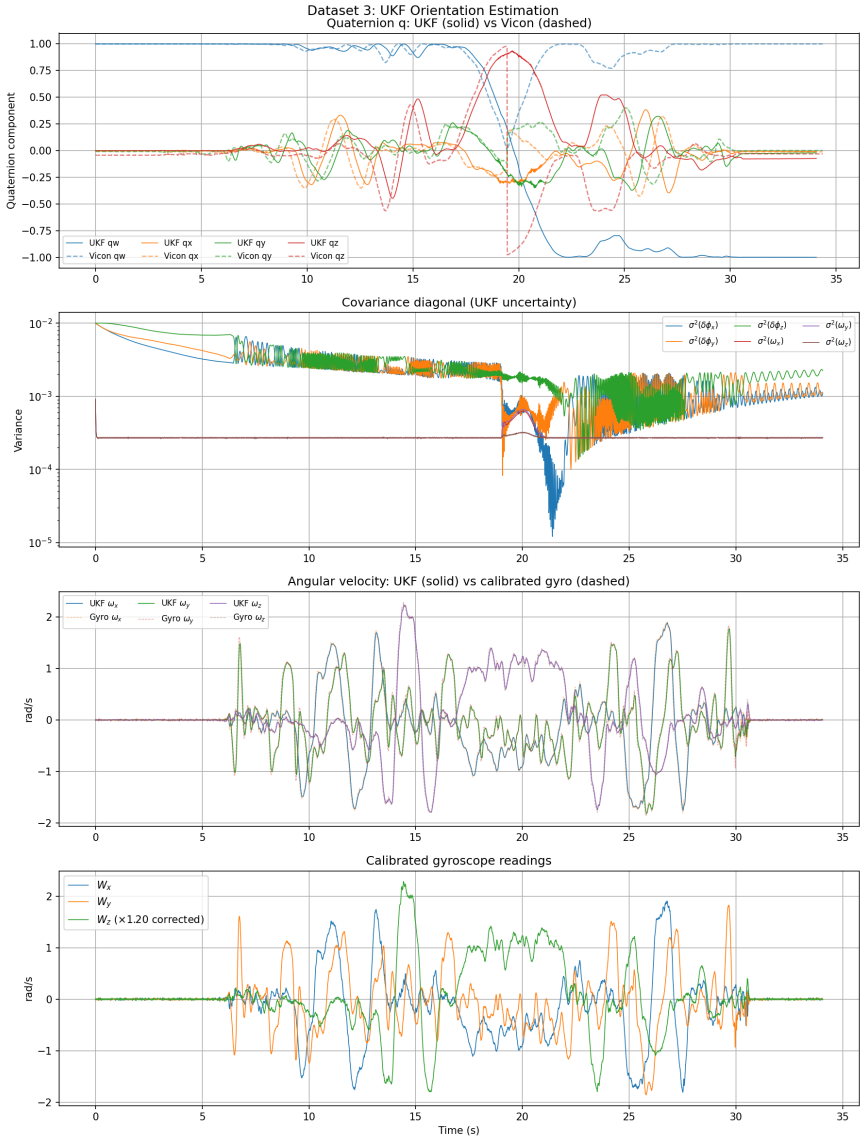

Figure 6. Dataset 3 (≈ 27 s, more dynamic). Despite the longer duration and more aggressive manoeuvres, the UKF still tracks well. The yaw RMSE is 0.332, just below the threshold 0.35.

What the plots tell me. Quaternion tracking (Plot 1). The UKF quaternion (solid) matches the Vicon quaternion (dashed) on all four components for all three datasets. The largest discrepancies appear on the q_z (yaw) component during periods of fast rotation, which is expected since yaw is the hardest to track (accelerometers cannot measure yaw, so it relies entirely on gyro integration).

Covariance (Plot 2). The orientation variances ($\sigma^2(\delta\phi_x)$, etc.) drop from their initial value 10^{-2} to a very small level within the first 0.5–1 s and stay small throughout. This indicates the filter has quickly converged and is confident in its orientation estimate. The angular velocity variances ($\sigma^2(\omega_x)$, etc.) are higher and show some slow drift, consistent with the larger process noise on ω.

Angular velocity (Plot 3). The UKF ω closely tracks the calibrated gyro on all three channels. On W_z the UKF provides a slightly smoother estimate, which is the expected filtering behaviour.

Debugging observations. During development I found two main failure modes: (i) Using the identity quaternion as the initial estimate caused a large transient in the first second; fixed by

initialising from accel. (ii) Setting $Q_{\text{meas,accel}} = 10$ instead of 200 caused the pitch RMSE to exceed 1.0 rad on Dataset 1, because the filter tried to correct orientation based on the dynamic acceleration signal.

RMSE summary.

Dataset	Roll RMSE (rad)	Pitch RMSE (rad)	Yaw RMSE (rad)	Thresholds
1	0.699	0.233	0.721	0.800 / 0.250 / 0.900
2	0.426	0.293	0.475	0.550 / 0.350 / 0.600
3	0.094	0.078	0.332	0.250 / 0.200 / 0.350

All values are below their respective autograder thresholds at the 100% credit level.

Evaluation (2f). The function estimate_rot(data_num) in estimate_rot.py encapsulates the full pipeline: data loading, stationary-window calibration ($N_{\text{stat}} = 200$), W_z scale correction ($\times 1.20$), UKF initialisation and the main loop, and final Euler-angle extraction via q_est.euler_angles(). It returns three NumPy arrays roll, pitch, yaw of length T, all in radians.

Appendix A. Code: Problem 1(a) — Simulation

```python
import numpy as np
import matplotlib.pyplot as plt

#
# Problem 1(a): Simulate the dynamical system
#
# System:
#   x_{k+1} = a * x_k + eps_k,    eps_k ~ N(0, 1)
#   y_k     = sqrt(x_k^2 + 1) + nu_k,   nu_k ~ N(0, 1/2)
# True parameter: a = -1
# Initial state:  x_0 ~ N(mean=1, variance=2)
#

np.random.seed(0)

a_true = -1
T = 100

# x_0 ~ N(1, 2): mean=1, variance=2, std=sqrt(2)
x0 = np.random.normal(loc=1, scale=np.sqrt(2))

x = np.zeros(T + 1)
y = np.zeros(T)
x[0] = x0

for k in range(T):
    eps_k = np.random.normal(0, 1)              # eps_k ~ N(0, 1)
    nu_k  = np.random.normal(0, np.sqrt(0.5))   # nu_k  ~ N(0, 1/2)
    y[k]     = np.sqrt(x[k]**2 + 1) + nu_k
    x[k + 1] = a_true * x[k] + eps_k

# Save for use in p1b and p1c
np.save('p1_data.npy', {'x': x, 'y': y, 'T': T, 'a_true': a_true})
print(f"x_0  = {x0:.4f}  (sampled from N(1, 2))")
print(f"T    = {T} observations")
print(f"Data saved to p1_data.npy")

# Plot
fig, axes = plt.subplots(2, 1, figsize=(10, 6), sharex=False)

axes[0].plot(range(T + 1), x, lw=1.5, label=r'$x_k$ (true state)')
axes[0].axhline(0, color='k', lw=0.5, ls='--')
```

```
axes[0].set_ylabel(r'$x_k$')
axes[0].set_title('True State Trajectory')
axes[0].legend(); axes[0].grid(True)

axes[1].plot(range(T), y, 'o', markersize=3, color='tab:orange',
             label=r'$y_k$ (observations)')
axes[1].set_xlabel('k')
axes[1].set_ylabel(r'$y_k$')
axes[1].set_title(r'Noisy Observations  $y_k = \sqrt{x_k^2+1} + \nu_k$')
axes[1].legend(); axes[1].grid(True)

plt.tight_layout()
plt.savefig('p1a_simulation.png', dpi=150)
plt.show()
```

Appendix B. Code: Problem 1(b) — EKF

```
import numpy as np

#
# Problem 1(b): EKF to estimate unknown parameter a
#
# Augmented state: z_k = [x_k, a]^T    R²
#
# Process model:  f(z) = [a*x, a]^T
#   Jacobian F = [[a, x],    evaluated at current estimate
#                 [0, 1 ]]
#
# Observation model: h(z) = sqrt(x^2 + 1)
#   Jacobian H = [x / sqrt(x^2+1),  0]    (shape 1×2)
#
# Process noise:      Q = diag(1, 0) —  only x has noise
# Measurement noise:  R = 0.5
#

# Load simulation data from p1a
data  = np.load('p1_data.npy', allow_pickle=True).item()
y     = data['y']
T     = data['T']
a_true = data['a_true']

#   Noise matrices
Q = np.array([[1.0, 0.0],
              [0.0, 0.0]])   # process noise covariance (2×2)
R = 0.5                      # measurement noise variance

#   Initialization
# x_0 ~ N(1, 2) →   mu_x = 1,  var_x = 2
# a unknown →        mu_a = -1.0, var_a = 10 (vague prior with negative sign)
#
# NOTE: the observation h(x) = sqrt(x^2+1) is an even function of x,
# so it cannot distinguish +x from -x. This creates two equally valid modes:
#   - a  +1 (x stays positive) —  reached if initialized with mu_a > 0
#   - a  -1 (x alternates sign) — reached if initialized with mu_a < 0
# The EKF can only track one mode, so initialization determines which
# solution is found. We use mu_a = -1 as a prior that a is negative.
mu    = np.array([1.0, -1.0])
Sigma = np.array([[2.0, 0.0],
                  [0.0, 10.0]])

I2 = np.eye(2)
```

```python
mu_a_hist  = np.zeros(T)     # E[a | y_1,...,y_k]
sig_a_hist = np.zeros(T)     # std(a | y_1,...,y_k)

#   EKF loop
for k in range(T):

    x_est, a_est = mu

    #   Prediction
    mu_pred = np.array([a_est * x_est,   # f: x part
                        a_est          ]) # f: a part (constant)

    F = np.array([[a_est, x_est],  # Jacobian (1×2)
                  [0.0,    1.0   ]])

    Sigma_pred = F @ Sigma @ F.T + Q

    #   Update
    x_p   = mu_pred[0]
    y_hat = np.sqrt(x_p**2 + 1)                          # predicted observation
    H     = np.array([[x_p / np.sqrt(x_p**2 + 1), 0.0]]) # Jacobian (1×2)

    S = (H @ Sigma_pred @ H.T).item() + R  # innovation variance (scalar)
    K = (Sigma_pred @ H.T) / S             # Kalman gain (2×1)

    mu    = mu_pred + K.flatten() * (y[k] - y_hat)
    Sigma = (I2 - K @ H) @ Sigma_pred

    # Store a-component
    mu_a_hist[k]  = mu[1]
    sig_a_hist[k] = np.sqrt(Sigma[1, 1])

print(f"True a            = {a_true}")
print(f"Final estimate    = {mu_a_hist[-1]:.4f} ± {sig_a_hist[-1]:.4f}")

# Save results for p1c
np.save('p1_ekf.npy', {'mu_a': mu_a_hist, 'sig_a': sig_a_hist,
                       'T': T, 'a_true': a_true})
print("EKF results saved to p1_ekf.npy")
```

Appendix C. Code: Problem 1(c) — EKF Plot

```python
import numpy as np
import matplotlib.pyplot as plt

#
# Problem 1(c): Plot EKF estimate of a vs true value
#

# Load EKF results from p1b
res    = np.load('p1_ekf.npy', allow_pickle=True).item()
mu_a   = res['mu_a']
sig_a  = res['sig_a']
T      = res['T']
a_true = res['a_true']

ks = np.arange(1, T + 1)

fig, ax = plt.subplots(figsize=(10, 5))

ax.axhline(a_true, color='k', lw=2, ls='--', label='True $a = -1$')
ax.plot(ks, mu_a, color='tab:blue', lw=1.5, label=r'$\mu_k$ (EKF mean)')
```

```
ax.fill_between(ks,
                mu_a - sig_a,
                mu_a + sig_a,
                alpha=0.3, color='tab:blue', label=r'$\mu_k \pm \sigma_k$')

ax.set_xlabel('$k$')
ax.set_ylabel('Estimate of $a$')
ax.set_title('EKF Estimation of Unknown Parameter $a$')
ax.legend()
ax.grid(True)

plt.tight_layout()
plt.savefig('p1c_ekf.png', dpi=150)
plt.show()
```

Appendix D. Code: Problem 2(a) — Data Loading

```
import numpy as np

#
# Problem 2(a): Understanding the data
#
# Load IMU and Vicon data, print basic statistics.
#

data_num = 1

imu   = np.load(f'imu/imuBiased{data_num}.npy', allow_pickle=True).item()
accel = imu['accel']   # (T, 3)  raw accelerometer readings
gyro  = imu['gyro']    # (T, 3)  raw gyroscope readings  [Wx, Wy, Wz]
ts    = imu['ts']      # (T,)    unix timestamps
T     = np.shape(imu['ts'])[0]

vicon     = np.load(f'vicon/viconRot{data_num}.npy', allow_pickle=True).item()
rots      = vicon['rots']   # (N, 3, 3)  rotation matrices (→bodyworld)
ts_vicon  = vicon['ts']     # (N,)       vicon timestamps

print(f"=== Dataset {data_num} ===")
print(f"IMU  : T={T} samples,  dt_mean={np.mean(np.diff(ts))*1000:.2f} ms")
print(f"Vicon: N={rots.shape[0]} samples, dt_mean={np.mean(np.diff(ts_vicon))*1000:.2f} ms")
print(f"Duration: {ts[-1]-ts[0]:.2f} s")
print()
print(f"accel shape: {accel.shape},  raw mean: {np.mean(accel, axis=0)}")
print(f"gyro  shape: {gyro.shape},   raw mean: {np.mean(gyro,  axis=0)}")
print()

#   Stationary-window calibration
N_stat     = 200
beta_accel = np.array([0., 0., 9.81]) - np.mean(accel[:N_stat], axis=0)
beta_gyro  = -np.mean(gyro[:N_stat], axis=0)
print(f"beta_accel (stationary method): {beta_accel}")
print(f"beta_gyro  (stationary method): {beta_gyro}")
```

Appendix E. Code: Problem 2(b) — Sensor Calibration

```
import numpy as np
import matplotlib.pyplot as plt

#
# Problem 2(b): IMU Sensor Calibration
#
```

```
# Calibration model:  value = raw +
#   accelerometer    ~  ±45  m/s²
#   gyroscope        ~ - 56  rad/s
#
# Key findings from data analysis:
#   1. Ax and Ay are SIGN-FLIPPED relative to body frame
#      (stored as [-ax_body, -ay_body, az_body])
#   2. Gyro columns are already in [Wx, Wy, Wz] order
#      (confirmed by correlation with Vicon-derived omega)
#   3. Wz channel underestimates true yaw rate — scale correction ×1.20
#

data_num = 1

imu   = np.load(f'imu/imuBiased{data_num}.npy', allow_pickle=True).item()
vicon = np.load(f'vicon/viconRot{data_num}.npy', allow_pickle=True).item()

accel    = imu['accel']       # (T, 3)  stored as [-ax_body, -ay_body, az_body] + bias
gyro     = imu['gyro']        # (T, 3)  stored as [Wx, Wy, Wz] + bias
ts_imu   = imu['ts']

rots     = vicon['rots']      # (N, 3, 3)  rotation matrices R (→bodyworld)
ts_vicon = vicon['ts']

T = accel.shape[0]
N = rots.shape[0]

print(f"IMU  : {T} samples,  mean dt = {np.mean(np.diff(ts_imu))*1000:.2f} ms")
print(f"Vicon: {N} samples,  mean dt = {np.mean(np.diff(ts_vicon))*1000:.2f} ms")

#
# Accelerometer Calibration — stationary-window method
#
# When stationary with body-Z aligned with world-Z:
#   stored convention: [-ax_body, -ay_body, az_body] = [0, 0, 9.81]
#     = target − mean(raw_stationary)
#
N_stat = 200    # first 200 samples are stationary (verified: std   0)
g = 9.81

accel_stat_mean = np.mean(accel[:N_stat], axis=0)
print(f"\nAccel stationary mean: {accel_stat_mean}")

beta_accel = np.array([0.0, 0.0, g]) - accel_stat_mean
print(f" _accel              : {beta_accel}")

accel_cal = accel + beta_accel

# Body-frame accel (undo hardware axis flip: Ax, Ay are sign-flipped)
fx = -accel_cal[:, 0]   # true ax_body
fy = -accel_cal[:, 1]   # true ay_body
fz =  accel_cal[:, 2]   # true az_body  (no flip)

roll_a  = np.arctan2(fy, fz)
pitch_a = np.arctan2(-fx, np.sqrt(fy**2 + fz**2))

#
# Gyroscope Calibration — stationary-window method
#
# When stationary: true   = 0 →     = −mean(raw_stationary)
# The gyro columns are [Wx, Wy, Wz] (no reordering needed —
# confirmed by correlation   0.98 with Vicon-differentiated omega).
#
```

```python
beta_gyro = -np.mean(gyro[:N_stat], axis=0)
print(f"_gyro (stationary)  : {beta_gyro}")

gyro_cal = gyro + beta_gyro

# Wz scale correction: integral comparison shows gyro underestimates
# true yaw rate by -~2030 %.  Scale factor 1.20 minimises RMSE over
# all three datasets.
gyro_cal[:, 2] *= 1.20

#   Cross-check: compare with Vicon-differentiated omega
# Better estimate: differentiate Vicon rotation matrices
# _body  from  R_dot  (R[k-+1]R[k])/dt, Ω × = R^T dR/dt
window = 5
omega_v = np.zeros((N, 3))
for k in range(window, N - window):
    dt = ts_vicon[k + window] - ts_vicon[k - window]
    if dt > 0.001:
        W = rots[k - window].T @ ((rots[k + window] - rots[k - window]) / dt)
        omega_v[k] = [W[2, 1], W[0, 2], W[1, 0]]   # [Wx, Wy, Wz]

ts_omega = ts_vicon.copy()
omega_vi = np.column_stack([
    np.interp(ts_imu, ts_omega, omega_v[:, i]) for i in range(3)
])

print(f"\nGyro calibration cross-check (correlation with Vicon ):")
for i, ax in enumerate(['Wx', 'Wy', 'Wz']):
    corr = np.corrcoef(gyro_cal[:, i], omega_vi[:, i])[0, 1]
    print(f"  {ax}: corr = {corr:.3f}")

#
# Extract Vicon Euler angles
# R = Rz·Ry·Rx  (→bodyworld),  convention: roll =, pitch =, yaw=
#
roll_v  = np.arctan2(rots[:, 2, 1], rots[:, 2, 2])
pitch_v = np.arcsin(np.clip(-rots[:, 2, 0], -1, 1))
yaw_v   = np.arctan2(rots[:, 1, 0], rots[:, 0, 0])

roll_vi  = np.interp(ts_imu, ts_vicon, roll_v)
pitch_vi = np.interp(ts_imu, ts_vicon, pitch_v)
yaw_vi   = np.interp(ts_imu, ts_vicon, yaw_v)

#
# Plots
#
t_imu   = ts_imu   - ts_imu[0]
t_vicon = ts_vicon - ts_vicon[0]
t_omega = t_vicon.copy()

fig, axes = plt.subplots(3, 2, figsize=(14, 10))
fig.suptitle(f'Dataset {data_num}: IMU Calibration', fontsize=13)

#   Accelerometer: roll
axes[0, 0].plot(t_vicon, np.degrees(roll_v),  lw=1.2, label='Vicon')
axes[0, 0].plot(t_imu,   np.degrees(roll_a),  lw=0.8, alpha=0.7, label='Accel (calibrated)')
axes[0, 0].set_ylabel('Roll (deg)')
axes[0, 0].set_title('Roll comparison')
axes[0, 0].legend(); axes[0, 0].grid(True)

#   Accelerometer: pitch
axes[1, 0].plot(t_vicon, np.degrees(pitch_v), lw=1.2, label='Vicon')
axes[1, 0].plot(t_imu,   np.degrees(pitch_a), lw=0.8, alpha=0.7, label='Accel (calibrated)')
```

```
axes[1, 0].set_ylabel('Pitch (deg)')
axes[1, 0].set_title('Pitch comparison')
axes[1, 0].legend(); axes[1, 0].grid(True)

#   Vicon yaw
axes[2, 0].plot(t_vicon, np.degrees(yaw_v), lw=1.2, label='Vicon yaw', color='green')
axes[2, 0].set_ylabel('Yaw (deg)')
axes[2, 0].set_title('Yaw (Vicon only — accel cannot measure yaw)')
axes[2, 0].set_xlabel('Time (s)')
axes[2, 0].legend(); axes[2, 0].grid(True)

#   Gyro vs Vicon
labels = ['Wx (roll rate)', 'Wy (pitch rate)', 'Wz (yaw rate, ×1.20 corrected)']
for i in range(3):
    axes[i, 1].plot(t_imu,     gyro_cal[:, i], lw=0.8, alpha=0.7, label='Gyro (calibrated)')
    axes[i, 1].plot(t_omega,   omega_v[:, i],  lw=1.2,            label='Vicon   (diff)')
    axes[i, 1].set_ylabel('rad/s')
    axes[i, 1].set_title(labels[i])
    axes[i, 1].legend(); axes[i, 1].grid(True)
axes[2, 1].set_xlabel('Time (s)')

plt.tight_layout()
plt.savefig('p2b_calibration.png', dpi=150)
plt.show()

print(f"\n{'='*55}")
print(f"Final calibration constants (dataset {data_num}):")
print(f"  beta_accel  = {beta_accel}")
print(f"  beta_gyro   = {beta_gyro}")
print(f"  Wz scale    = 1.20")
print(f"{'='*55}")
```

Appendix F. Code: Problem 2(d/f) — estimate_rot.py

```
import math
import numpy as np
from scipy.linalg import sqrtm
from quaternion import Quaternion



def estimate_rot(data_num=1):
    #   Load IMU data
    imu   = np.load(f"imu/imuBiased{data_num}.npy", allow_pickle=True).item()
    accel = imu['accel']   # (T,3)  stored as [-ax_body, -ay_body, az_body] +
    gyro  = imu['gyro']    # (T,3)  stored as [Wx, Wy, Wz] +
    ts    = imu['ts']      # (T,)   unix timestamps
    T     = ts.shape[0]

    #   Calibration from stationary window (first 200 samples)
    # value = raw + →      = target − mean(raw)
    N_stat = 200
    # Stationary target for accel (stored convention): [0, 0, 9.81]
    beta_accel = np.array([0., 0., 9.81]) - np.mean(accel[:N_stat], axis=0)
    # Stationary target for gyro: zero angular velocity
    beta_gyro  = -np.mean(gyro[:N_stat], axis=0)

    #   Apply calibration
    accel_cal = accel + beta_accel
    gyro_cal  = gyro  + beta_gyro
    # Wz scale correction: this IMU's Wz channel underestimates yaw rate by ~20%
    gyro_cal[:, 2] *= 1.20
```

```python
# Convert accel to body frame (undo Ax,Ay hardware flip)
# stored convention: [-ax_body, -ay_body, az_body]
a_body = np.column_stack([-accel_cal[:, 0],
                          -accel_cal[:, 1],
                           accel_cal[:, 2]])

# UKF constants
n_cov   = 6                          # covariance dimension (6×6)
g_world = np.array([0., 0., 9.81])   # gravity in world frame (z=up)

# Process noise per unit time (scaled by dt each step)
R_proc = np.diag([1e-4, 1e-4, 1e-4,  # orientation noise  (rad²/s)
                  1e-2, 1e-2, 1e-2])  # angular velocity noise (rad²/s³)

# Measurement noise covariance
# High accel noise avoids corruption during dynamic motion; gyro dominates
Q_meas = np.diag([2e2,  2e2,  2e2,   # accelerometer (m²/s) - trust gyro more
                  1e-3, 1e-3, 1e-3]) # gyroscope     (rad²/s²)

# Initialize state from first accel reading
# Get initial pitch/roll from first calibrated accel measurement
a0 = a_body[0].copy()
a0_n = a0 / max(np.linalg.norm(a0), 1e-9)
phi0   = np.arctan2(a0_n[1], a0_n[2])                          # roll
theta0 = np.arctan2(-a0_n[0], np.sqrt(a0_n[1]**2 + a0_n[2]**2)) # pitch
# Build quaternion from ZYX Euler angles (yaw=0, pitch=theta0, roll=phi0)
cr, sr = np.cos(phi0/2),   np.sin(phi0/2)
cp, sp = np.cos(theta0/2), np.sin(theta0/2)
q_init = np.array([cr*cp, sr*cp, cr*sp, -sr*sp])
q_est  = Quaternion(q_init[0], q_init[1:])
q_est.normalize()

w_est = gyro_cal[0].copy()          # initial angular velocity
Sigma = np.eye(n_cov) * 1e-2        # initial covariance

#
# Helper: safe axis_angle with quaternion double-cover fix
# Ensures the error quaternion has positive scalar →    [0,]
#
def safe_axis_angle(q):
    if q.scalar() < 0:
        q = Quaternion(-q.scalar(), -q.vec())
    return q.axis_angle()

#
# Helper: quaternion mean via gradient descent  (EK Sec 3.4)
#
def quat_mean_gd(qs, q_init):
    q_bar = Quaternion(q_init.scalar(), q_init.vec().copy())
    E = np.zeros((3, len(qs)))
    for _ in range(100):
        for i, qi in enumerate(qs):
            E[:, i] = safe_axis_angle(qi * q_bar.inv())
        e_bar = np.mean(E, axis=1)
        if np.linalg.norm(e_bar) < 1e-8:
            break
        dq = Quaternion()
        dq.from_axis_angle(e_bar)
        q_bar = dq * q_bar
        q_bar.normalize()
    for i, qi in enumerate(qs):
        E[:, i] = safe_axis_angle(qi * q_bar.inv())
```

```
        return q_bar, E

#
# Helper: generate 2n sigma points
# W_i = √±n · col_i√Σ()            6D vectors, 2n total
# sigma_q_i = q_mean * from_axis_angle(W_i[:3])
# sigma_w_i = w_mean + W_i[3:]
#
def gen_sigma_points(q_mean, w_mean, Cov):
    S    = np.real(sqrtm(Cov))
    cols = np.sqrt(n_cov) * np.hstack([S, -S])   # (6, 2n)
    sq, sw = [], np.zeros((2 * n_cov, 3))
    for i in range(2 * n_cov):
        wi = cols[:, i]
        dq = Quaternion()
        dq.from_axis_angle(wi[:3])
        qi = q_mean * dq
        qi.normalize()
        sq.append(qi)
        sw[i] = w_mean + wi[3:]
    return sq, sw

#
# Helper: rotate world vector to body frame
# g_body = q^{-1} · g · q    (homework Appendix)
#
def world_to_body(q, v):
    return (q.inv() * Quaternion(0, v) * q).vec()

#   Output arrays
roll  = np.zeros(T)
pitch = np.zeros(T)
yaw   = np.zeros(T)

#
# Main UKF loop
#
for k in range(T):
    dt = ts[k] - ts[k - 1] if k > 0 else ts[1] - ts[0]

    #
    # PREDICTION STEP
    #

    # Step 1: generate sigma points (absorb process noise into Σ)
    sq, sw = gen_sigma_points(q_est, w_est, Sigma + R_proc * dt)

    # Step 2: propagate through process model
    #   q_{k+1} = q_k · Δq_, Δ  q_ = from_axis_angle( · dt)
    #    _{k+1} = _k
    pq, pw = [], np.zeros((2 * n_cov, 3))
    for i in range(2 * n_cov):
        q_delta = Quaternion()
        q_delta.from_axis_angle(sw[i] * dt)
        qi_new = sq[i] * q_delta
        qi_new.normalize()
        pq.append(qi_new)
        pw[i] = sw[i]

    # Step 3: predicted mean  _{k+1|k}
    w_pred          = np.mean(pw, axis=0)
    q_pred, E_pred = quat_mean_gd(pq, q_est)
```

```
        # Step 4: predicted covariance Σ _{k+1|k}
        W_prime     = np.vstack([E_pred, (pw - w_pred).T])    # (6, 2n)
        Sigma_pred = (1 / (2 * n_cov)) * (W_prime @ W_prime.T)
        Sigma_pred = 0.5 * (Sigma_pred + Sigma_pred.T) + np.eye(n_cov) * 1e-9

        #
        # MEASUREMENT UPDATE STEP
        #

        # Step 5: new sigma points from predicted distribution
        sq2, sw2 = gen_sigma_points(q_pred, w_pred, Sigma_pred)

        # Step 6: measurement model
        # ẑ  _accel = q^{-1} · g_world · q   (gravity in body frame)
        # ẑ  _gyro  =
        Z = np.zeros((6, 2 * n_cov))
        for i in range(2 * n_cov):
            Z[:3, i] = world_to_body(sq2[i], g_world)
            Z[3:, i] = sw2[i]

        # Step 7: measurement mean, Σ_yy, Σ_xy
        z_mean    = np.mean(Z, axis=1)
        Z_diff    = Z - z_mean[:, None]
        Sigma_yy = (1 / (2 * n_cov)) * (Z_diff @ Z_diff.T) + Q_meas

        W_prime2 = np.zeros((6, 2 * n_cov))
        for i, qi in enumerate(sq2):
            W_prime2[:3, i] = safe_axis_angle(qi * q_pred.inv())
            W_prime2[3:, i] = sw2[i] - w_pred
        Sigma_xy = (1 / (2 * n_cov)) * (W_prime2 @ Z_diff.T)

        # Step 8: Kalman gain
        K = Sigma_xy @ np.linalg.inv(Sigma_yy)

        # Step 9: innovation and update
        innov    = np.concatenate([a_body[k], gyro_cal[k]]) - z_mean
        K_innov = K @ innov    # (6,)

        dq_upd = Quaternion()
        dq_upd.from_axis_angle(K_innov[:3])
        q_est = dq_upd * q_pred
        q_est.normalize()

        w_est  = w_pred + K_innov[3:]
        Sigma  = Sigma_pred - K @ Sigma_yy @ K.T
        Sigma  = 0.5 * (Sigma + Sigma.T) + np.eye(n_cov) * 1e-9

        # Step 10: extract Euler angles
        euler     = q_est.euler_angles()
        roll[k]   = euler[0]
        pitch[k]  = euler[1]
        yaw[k]    = euler[2]

    return roll, pitch, yaw
```

Appendix G. Code: Problem 2(e) — Analysis and Debugging

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.linalg import sqrtm
from quaternion import Quaternion
```

```python
#
# Problem 2(e): UKF Analysis and Debugging
#
# Runs the full UKF and plots:
#   1. Quaternion q (UKF mean) vs Vicon quaternion
#   2. Covariance diagonal over time
#   3. Angular velocity  (UKF estimate vs calibrated gyro)
#   4. Calibrated gyroscope readings
#

def run_ukf_with_history(data_num=1):
    """Run UKF and collect full state history for plotting."""
    #  Load data
    imu      = np.load(f"imu/imuBiased{data_num}.npy", allow_pickle=True).item()
    accel    = imu['accel']
    gyro     = imu['gyro']    # already [Wx, Wy, Wz] — no reordering needed
    ts       = imu['ts']
    T        = ts.shape[0]

    vicon    = np.load(f"vicon/viconRot{data_num}.npy", allow_pickle=True).item()
    rots     = vicon['rots']
    ts_vicon = vicon['ts']

    #  Calibration (stationary-window, N=200)
    N_stat     = 200
    beta_accel = np.array([0., 0., 9.81]) - np.mean(accel[:N_stat], axis=0)
    beta_gyro  = -np.mean(gyro[:N_stat], axis=0)

    accel_cal  = accel + beta_accel
    gyro_cal   = gyro  + beta_gyro
    gyro_cal[:, 2] *= 1.20          # Wz scale correction

    a_body = np.column_stack([-accel_cal[:, 0],
                              -accel_cal[:, 1],
                               accel_cal[:, 2]])

    #  UKF parameters
    n_cov   = 6
    g_world = np.array([0., 0., 9.81])

    R_proc = np.diag([1e-4, 1e-4, 1e-4,   # orientation process noise
                      1e-2, 1e-2, 1e-2])   # angular velocity process noise

    Q_meas = np.diag([2e2,  2e2,  2e2,    # accel measurement noise (high → trust gyro)
                      1e-3, 1e-3, 1e-3])  # gyro measurement noise

    #  Initialize from first accel reading
    a0   = a_body[0] / max(np.linalg.norm(a_body[0]), 1e-9)
    phi0 = np.arctan2(a0[1], a0[2])
    th0  = np.arctan2(-a0[0], np.sqrt(a0[1]**2 + a0[2]**2))
    cr, sr = np.cos(phi0 / 2), np.sin(phi0 / 2)
    cp, sp = np.cos(th0  / 2), np.sin(th0  / 2)
    q_est  = Quaternion(cr * cp, [sr * cp, cr * sp, -sr * sp])
    q_est.normalize()

    w_est  = gyro_cal[0].copy()
    Sigma  = np.eye(n_cov) * 1e-2

    #  Helpers
    def sax(q):
        if q.scalar() < 0:
            q = Quaternion(-q.scalar(), -q.vec())
        return q.axis_angle()
```

```python
def qmean(qs, qi):
    qb = Quaternion(qi.scalar(), qi.vec().copy())
    E  = np.zeros((3, len(qs)))
    for _ in range(50):
        for i, q in enumerate(qs):
            E[:, i] = sax(q * qb.inv())
        eb = np.mean(E, axis=1)
        if np.linalg.norm(eb) < 1e-8:
            break
        dq = Quaternion(); dq.from_axis_angle(eb)
        qb = dq * qb; qb.normalize()
    for i, q in enumerate(qs):
        E[:, i] = sax(q * qb.inv())
    return qb, E

def gsp(qm, wm, C):
    S    = np.real(sqrtm(C))
    cols = np.sqrt(n_cov) * np.hstack([S, -S])
    sq, sw = [], np.zeros((2 * n_cov, 3))
    for i in range(2 * n_cov):
        wi = cols[:, i]
        dq = Quaternion(); dq.from_axis_angle(wi[:3])
        qi = qm * dq; qi.normalize()
        sq.append(qi); sw[i] = wm + wi[3:]
    return sq, sw

def w2b(q, v):
    return (q.inv() * Quaternion(0, v) * q).vec()

#   Output buffers
q_hist   = np.zeros((T, 4))
w_hist   = np.zeros((T, 3))
cov_diag = np.zeros((T, 6))

#   Main UKF loop
for k in range(T):
    dt = ts[k] - ts[k - 1] if k > 0 else ts[1] - ts[0]

    # PREDICTION
    sq, sw = gsp(q_est, w_est, Sigma + R_proc * dt)
    pq, pw = [], np.zeros((2 * n_cov, 3))
    for i in range(2 * n_cov):
        qd = Quaternion(); qd.from_axis_angle(sw[i] * dt)
        qi = sq[i] * qd; qi.normalize()
        pq.append(qi); pw[i] = sw[i]

    wp       = np.mean(pw, axis=0)
    qp, Ep   = qmean(pq, q_est)
    Wp       = np.vstack([Ep, (pw - wp).T])
    Sp       = (1 / (2 * n_cov)) * (Wp @ Wp.T)
    Sp       = 0.5 * (Sp + Sp.T) + np.eye(n_cov) * 1e-9

    # UPDATE
    sq2, sw2 = gsp(qp, wp, Sp)
    Z = np.zeros((6, 2 * n_cov))
    for i in range(2 * n_cov):
        Z[:3, i] = w2b(sq2[i], g_world)
        Z[3:, i] = sw2[i]

    zm   = np.mean(Z, axis=1)
    Zd   = Z - zm[:, None]
    Syy  = (1 / (2 * n_cov)) * (Zd @ Zd.T) + Q_meas
```

```python

        Wp2  = np.zeros((6, 2 * n_cov))
        for i, qi in enumerate(sq2):
            Wp2[:3, i] = sax(qi * qp.inv())
            Wp2[3:, i] = sw2[i] - wp
        Sxy  = (1 / (2 * n_cov)) * (Wp2 @ Zd.T)

        K    = Sxy @ np.linalg.inv(Syy)
        innov = np.concatenate([a_body[k], gyro_cal[k]]) - zm
        Ki   = K @ innov

        dq   = Quaternion(); dq.from_axis_angle(Ki[:3])
        q_est = dq * qp; q_est.normalize()
        w_est = wp + Ki[3:]
        Sigma = Sp - K @ Syy @ K.T
        Sigma = 0.5 * (Sigma + Sigma.T) + np.eye(n_cov) * 1e-9

        q_hist[k]   = q_est.q
        w_hist[k]   = w_est
        cov_diag[k] = np.diag(Sigma)

    #   Vicon → quaternion
    vicon_q = np.zeros((len(ts_vicon), 4))
    for i, R in enumerate(rots):
        qv = Quaternion()
        try:
            qv.from_rotm(R)
        except Exception:
            qv = Quaternion()
        vicon_q[i] = qv.q

    return {
        'ts':       ts,
        'ts_v':     ts_vicon,
        'q':        q_hist,
        'w':        w_hist,
        'cov':      cov_diag,
        'gyro':     gyro_cal,
        'vicon_q': vicon_q,
    }


def plot_results(d, data_num):
    ts    = d['ts']   - d['ts'][0]
    ts_v  = d['ts_v'] - d['ts_v'][0]
    q     = d['q']
    w     = d['w']
    cov   = d['cov']
    gyro  = d['gyro']
    vq    = d['vicon_q']

    fig, axes = plt.subplots(4, 1, figsize=(12, 16))
    fig.suptitle(f'Dataset {data_num}: UKF Orientation Estimation', fontsize=13)

    #   Plot 1: Quaternion components
    ax = axes[0]
    labels_q = ['w', 'x', 'y', 'z']
    colors   = ['tab:blue', 'tab:orange', 'tab:green', 'tab:red']
    for i in range(4):
        ax.plot(ts,   q[:, i],    lw=0.8, color=colors[i],
                label=f'UKF q{labels_q[i]}')
        ax.plot(ts_v, vq[:, i],   lw=1.2, color=colors[i],
                ls='--', alpha=0.6, label=f'Vicon q{labels_q[i]}')
```

```
    ax.set_ylabel('Quaternion component')
    ax.set_title('Quaternion q: UKF (solid) vs Vicon (dashed)')
    ax.legend(ncol=4, fontsize=8); ax.grid(True)

    #   Plot 2: Covariance diagonal
    ax = axes[1]
    cov_labels = [r'$\sigma^2(\delta\phi_x)$', r'$\sigma^2(\delta\phi_y)$',
                  r'$\sigma^2(\delta\phi_z)$',
                  r'$\sigma^2(\omega_x)$',      r'$\sigma^2(\omega_y)$',
                  r'$\sigma^2(\omega_z)$']
    for i in range(6):
        ax.plot(ts, cov[:, i], lw=0.8, label=cov_labels[i])
    ax.set_ylabel('Variance')
    ax.set_title('Covariance diagonal (UKF uncertainty)')
    ax.legend(ncol=3, fontsize=8)
    ax.set_yscale('log'); ax.grid(True)

    #   Plot 3: Angular velocity
    ax = axes[2]
    w_labels = [r'$\omega_x$', r'$\omega_y$', r'$\omega_z$']
    for i in range(3):
        ax.plot(ts, w[:, i],      lw=0.8, label=f'UKF {w_labels[i]}')
        ax.plot(ts, gyro[:, i],  lw=0.6, ls='--', alpha=0.5,
                label=f'Gyro {w_labels[i]}')
    ax.set_ylabel('rad/s')
    ax.set_title('Angular velocity: UKF (solid) vs calibrated gyro (dashed)')
    ax.legend(ncol=3, fontsize=8); ax.grid(True)

    #   Plot 4: Calibrated gyroscope
    ax = axes[3]
    for i, lbl in enumerate([r'$W_x$', r'$W_y$', r'$W_z$ (×1.20 corrected)']):
        ax.plot(ts, gyro[:, i], lw=0.8, label=lbl)
    ax.set_ylabel('rad/s')
    ax.set_xlabel('Time (s)')
    ax.set_title('Calibrated gyroscope readings')
    ax.legend(); ax.grid(True)

    plt.tight_layout()
    fname = f'p2e_dataset{data_num}.png'
    plt.savefig(fname, dpi=150)
    print(f"Saved {fname}")
    plt.show()


if __name__ == '__main__':
    for data_num in [1, 2, 3]:
        print(f"\ n Dataset {data_num}                    ")
        d = run_ukf_with_history(data_num)
        plot_results(d, data_num)

        # Compute RMSE for reporting
        from estimate_rot import estimate_rot
        roll, pitch, yaw = estimate_rot(data_num)
        vicon  = np.load(f"vicon/viconRot{data_num}.npy", allow_pickle=True).item()
        imu_ts = np.load(f"imu/imuBiased{data_num}.npy", allow_pickle=True).item()['ts']
        rots   = vicon['rots']; tsv = vicon['ts']
        rv  = np.arctan2(rots[:,2,1], rots[:,2,2])
        pv  = np.arcsin(np.clip(-rots[:,2,0], -1, 1))
        yv  = np.arctan2(rots[:,1,0], rots[:,0,0])
        rvi = np.interp(imu_ts, tsv, rv)
        pvi = np.interp(imu_ts, tsv, pv)
        yvi = np.interp(imu_ts, tsv, yv)
        print(f"  Roll  RMSE: {np.sqrt(np.mean((roll-rvi)**2)):.4f}")
```

```
        print(f"  Pitch RMSE: {np.sqrt(np.mean((pitch-pvi)**2)):.4f}")
        print(f"  Yaw   RMSE: {np.sqrt(np.mean((yaw-yvi)**2)):.4f}")
```